

Questions

- 1) Cosa si intende per database?
- 2) Cos'è un DBMS?
- 3) Indica le principali clausole di uno statement SELECT in ordine di esecuzione logica. Descrivi per ciascuna delle clausole indicate la logica di funzionamento.
- 4) Descrivi, immaginando uno scenario a te familiare, il concetto di group by. Utilizza l'approccio che ritieni più efficiente per trasmettere il concetto (suggerimento: disegna anche una sola tabella in Excel o in word con poche colonne e pochi record e descrivi, basandosi sulla tabella stessa, un esempio di group by).
- 5) Descrivi la differenza tra uno schema OLTP e uno schema OLAP.
- 6) Dato un medesimo scenario di analisi, qual è la differenza in termini di risultato ottenibile tra una join e una subquery?
- 7) Cosa si intende per DML e DDL?
- 8) Quali istruzioni possono utilizzare per estrarre l'anno da un campo data? Proponi degli esempi.
- 9) Qual è la differenza tra gli operatori logici AND e OR?
- 10) È possibile innestare una query nella clausola SELECT?
- 11) Qual è la differenza tra l'operatore logico OR e l'operatore logico IN?
- 12) L'operatore logico BETWEEN include anche gli estremi del range specificato?

Risposte:

- 1) Una raccolta organizzata di dati strutturati in tabelle in relazione tra loro.
- 2) E' un sistema software progettato per consentire la creazione, la manipolazione e l'interrogazione di database.
- 3) Le principali clausole di una **SELECT** sono : **FROM** che seleziona la tabella da cui prendere i dati. **WHERE** che filtra le righe in base ad una condizione. **GROUP BY** che consente di creare dei gruppi (o raggruppamenti) per ogni combinazione univoca dei campi indicati nella group by list. **HAVING** che filtra i gruppi creati dalla clausola GROUP BY. **ORDER BY** . **SELECT** consente di selezionare le colonne.
- 4) Immaginando il db di una palestra, e una tabella IscrizioneCorsi con le colonne SocioID, Nome, Cognome, TipoCorso, QuotaMensile, e di voler conoscere quanti iscritti e quanto guadagno c'è per ogni corso. La query sarà : `SELECT TipoCorso, COUNT(SocioID), SUM(QuotaMensile) FROM IscrizioneCorsi GROUP BY TipoCorso.`

IscrizioneCorsi					Risultato GROUP BY		
SocioID	Nome	Cognome	TipoCorso	QuotaMensile	TipoCorso	Iscritti (Count)	Incasso (TOT)
1	Marco	Rossi	Pilates	50	Pilates	2	100
2	Elena	Verdi	Sala Pesi	40	Sala Pesi	2	80
3	Fabrizio	Gialli	Sala Pesi	40			
4	Ciro	Neri	Pilates	50	Crossfit	1	60
5	Giulia	Verdi	Crossfit	60			

- 5) Uno schema OLTP gestisce i dati in tempo reale mentre lo schema OLAP serve per gestire i dati "storici" accumulati nel tempo e quindi permette di fare analisi.
- 6) La differenza è che la JOIN unisce le tabelle permettendo di mostrare le colonne di entrambe nel risultato finale, mentre la SUBQUERY filtra o calcola i valori mantenendo visibili solo le colonne della tabella principale.
- 7) Per DML si intendono i comandi per creare o modificare il db (es. CREATE) mentre il DDL include i comandi per modificare la struttura del db (es. INSERT, UPDATE ecc.)
- 8) Possiamo utilizzare YEAR. Esempi : `SELECT YEAR(DataNascita) FROM Iscritti, SELECT YEAR(DataInizio) FROM IscrizioneCorsi.`

- 9) AND restituisce un valore solo se tutte le condizioni sono vere mentre OR restituisce un valore anche se almeno una condizione è vera.
- 10) Sì, con una subquery.
- 11) OR restituisce valori anche se solo una delle condizioni è vera mentre AND solo se tutte le condizioni sono vere

Case Study

ToysGroup è un'azienda che distribuisce articoli (giocattoli) in diverse aree geografiche del mondo.

I prodotti sono classificati in categorie e i mercati di riferimento dell'azienda sono classificati in regioni di vendita.

In particolare:

- 1) Le entità individuabili in questo scenario sono le seguenti:
 - Product
 - Region
 - Sales
- 2) Le relazioni tra le entità possono essere descritte nel modo seguente:
 - Product e Sales
 - Un prodotto può essere venduto tante volte (o nessuna) per cui è contenuto in una o più transazioni di vendita.
 - Ciascuna transazione di vendita è riferita ad uno solo prodotto
 - Region e Sales
 - Possono esserci molte o nessuna transazione per ciascuna regione
 - Ciascuna transazione di vendita è riferita ad una sola regione
- 3) Le entità Product e Region presentano delle gerarchie:
 - L'entità prodotto contiene, oltre alle informazioni del singolo prodotto, anche la descrizione della categoria di appartenenza. L'entità prodotto contiene quindi una gerarchia: un prodotto può appartenere ad una sola categoria mentre la stessa categoria può essere associata a molti prodotti diversi.
Esempio: gli articoli 'Bikes-100' e 'Bikes-200' appartengono alla categoria Bikes; gli articoli 'Bike Glove M' e 'Bike Gloves L' sono classificati come Clothing.
 - L'entità regione contiene una gerarchia: più stati sono classificati in una stessa regione di vendita e una stessa regione di vendita include molti stati.
Esempio: gli stati 'France' e 'Germany' sono classificati nella region WestEurope; gli stati 'Italy' e 'Greece' sono classificati nel mercato SouthEurope.

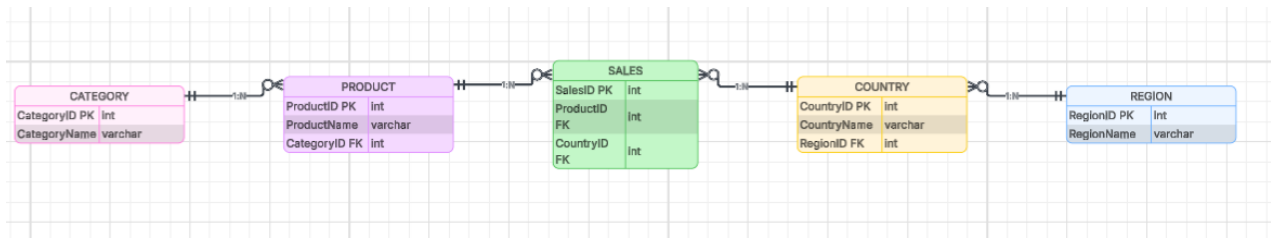
È necessario progettare e implementare fisicamente un database che modelli lo scenario garantendo l'**integrità referenziale** e la **minimizzazione della ridondanza dei dati**.

In altre parole, progetta opportunamente un numero di tabelle e di relazioni tra queste sufficiente a garantire la **consistenza del dato**.

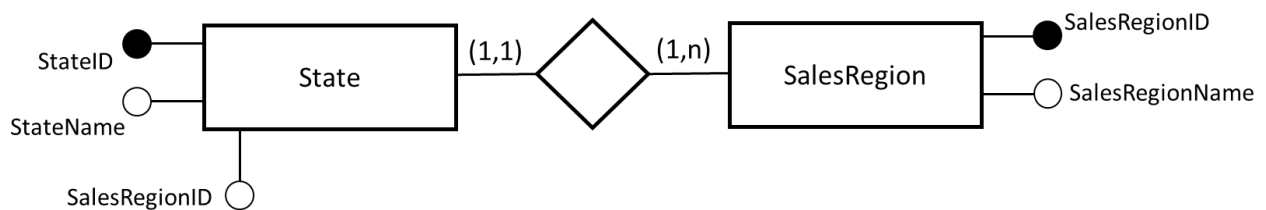
Task 1: Proponi una progettazione concettuale e logica della base dati

La progettazione concettuale deve includere tutte le entità coinvolte e le relazioni tra queste. Per ciascuna entità indica l'attributo chiave e i principali attributi descrittivi (non è necessario indicare tutti gli attributi).

Risposta task 1 :



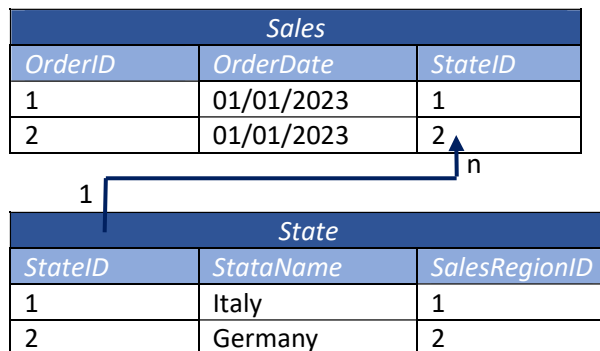
Esempio di schema E/R



Lo schema proposto è puramente esemplificativo e non esaustivo o completo per la soluzione!

La progettazione logica deve includere, per ciascuna tabella, tutte le colonne che poi verranno implementate fisicamente e deve esplicitare la cardinalità dei campi utilizzati per definire la relazione.

Esempio di schema grafico delle tabelle e delle relazioni tra le stesse.



Il diagramma è puramente esemplificativo e non esaustivo o completo per la soluzione!

Creazione DB ToysGroup :

```
create database ToysGroup;
use ToysGroup;
```

```
create table Category (CategoryID int primary key auto_increment, CategoryName varchar(50));
create table Region (RegionID int primary key auto_increment, RegionName varchar(50));
create table Product (ProductID int primary key auto_increment, ProductName varchar(50), CategoryID int,
foreign key (CategoryID) references Category (CategoryID));
create table Country (CountryID int primary key auto_increment, CountryName varchar(50), RegionID int,
```

```
foreign key (RegionID) references Region (RegionID));
create table Sales (SalesID int primary key auto_increment, ProductID int, CountryID int, SaleDate date,
Quantity int, UnitPrice decimal(10,2), foreign key (ProductID) references Product (ProductID), foreign key
(CountryID) references Country (CountryID));
```

```
insert into Category (CategoryName) values ('Biciclette'), ('Abbigliamento'), ('Accessori'), ('Componenti'),
('Caschi'), ('Guanti'), ('Scarpe'), ('Borse'), ('Luci'), ('Lucchetti');
```

```
insert into Region (RegionName) values ('EuropaOccidentale'), ('EuropaMeridionale'),
('EuropaSettentrionale'), ('AmericaDelNord'), ('AmericaDelSud'), ('AsiaOrientale'), ('AsiaOccidentale'),
('Africa'), ('Oceania'), ('EuropaOrientale');
```

```
insert into Product (ProductName, CategoryID) values ('Bici-100', 1), ('Bici-200', 1), ('Guanti M', 6), ('Guanti
L', 6), ('Casco Pro', 5), ('Casco Junior', 5), ('Scarpa Strada', 7), ('Scarpa Trail', 7), ('Borsa Città', 8), ('Giacca
Gara', 2);
```

```
insert into Country (CountryName, RegionID) values ('Francia', 1), ('Germania', 1), ('Italia', 2), ('Grecia', 2),
('Svezia', 3), ('Norvegia', 3), ('USA', 4), ('Canada', 4), ('Brasile', 5), ('Giappone', 6);
```

```
insert into Sales (ProductID, CountryID, SaleDate, Quantity, UnitPrice) values (1, 1, '2023-01-15', 10,
599.99), (2, 3, '2023-03-22', 5, 799.99), (3, 7, '2023-05-10', 20, 29.99), (4, 2, '2023-07-04', 15, 34.99), (5, 5,
'2023-09-18', 8, 149.99), (6, 9, '2024-01-08', 12, 129.99), (7, 4, '2024-03-15', 6, 89.99), (8, 6, '2024-05-20', 9,
94.99), (9, 10, '2024-08-11', 4, 49.99), (10, 8, '2024-11-30', 7, 199.99);
```

Task 2: Descrivi la struttura delle tabelle che reputi utili e sufficienti a modellare lo scenario proposto tramite la sintassi DDL. Implementa fisicamente le tabelle utilizzando il DBMS SQL Server(o altro).;

Risposta Task 2

```
create database ToysGroup;
```

```
use ToysGroup;
```

```
create table Category (CategoryID int primary key auto_increment, CategoryName varchar(50));
```

```
create table Region ( RegionID int primary key auto_increment, RegionName varchar (50));
```

```
create table Product ( ProductID int primary key auto_increment, ProductName varchar
(50),CategoryID int,foreign key (CategoryID) references Category (CategoryID));
```

```
create table Country (CountryID int primary key auto_increment, CountryName
varchar(50),RegionID int, foreign key(RegionID) references Region (RegionID));
```

```
create table Sales (SalesID int primary key auto_increment, ProductID int, CountryID int, foreign key
(ProductID) references Product (ProductID), foreign key (CountryID) references Country (CountryID));
```

Task 3: Popola le tabelle utilizzando dati a tua discrezione (sono sufficienti pochi record per tabella; riporta le query utilizzate)

Risposta Task 3

```
insert into Category (CategoryName) values ('Biciclette'), ('Abbigliamento'), ('Accessori'), ('Componenti'), ('Caschi'), ('Guanti'), ('Scarpe'), ('Borse'), ('Luci'), ('Lucchetti');
```

```
insert into Region (RegionName) values ('EuropaOccidentale'), ('EuropaMeridionale'), ('EuropaSettentrionale'), ('AmericaDelNord'), ('AmericaDelSud'), ('AsiaOrientale'), ('AsiaOccidentale'), ('Africa'), ('Oceania'), ('EuropaOrientale');
```

```
insert into Product (ProductName, CategoryID) values ('Bici-100', 1), ('Bici-200', 1), ('Guanti M', 6), ('Guanti L', 6), ('Casco Pro', 5), ('Casco Junior', 5), ('Scarpa Strada', 7), ('Scarpa Trail', 7), ('Borsa Città', 8), ('Giacca Gara', 2);
```

```
insert into Country (CountryName, RegionID) values ('Francia', 1), ('Germania', 1), ('Italia', 2), ('Grecia', 2), ('Svezia', 3), ('Norvegia', 3), ('USA', 4), ('Canada', 4), ('Brasile', 5), ('Giappone', 6);
```

```
insert into Sales (ProductID, CountryID) values (1, 1), (2, 3), (3, 7), (4, 2), (5, 5), (6, 9), (7, 4), (8, 6), (9, 10), (10, 8);
```

Task 4: Dopo aver popolate le tabelle, scrivi delle query utili a:

- 1) Verificare che i campi definiti come PK siano univoci. In altre parole, scrivi una query per determinare l'univocità dei valori di ciascuna PK (una query per tabella implementata).

Risposta 1

```
select count(*) as totalerighe, count(distinct categoryid) as valoriunicipk from category;
```

```
select count(*) as totalerighe, count(distinct regionid) as valoriunicipk from region;
```

```
select count(*) as totalerighe, count(distinct productid) as valoriunicipk from product;
```

```
select count(*) as totalerighe, count(distinct countryid) as valoriunicipk from country;
```

```
select count(*) as totalerighe, count(distinct salesid) as valoriunicipk from sales;
```

- 2) Esporre l'elenco delle transazioni indicando nel result set il codice documento, la data, il nome del prodotto, la categoria del prodotto, il nome dello stato, il nome della regione di vendita e un campo booleano valorizzato in base alla condizione che siano passati più di 180 giorni dalla data vendita o meno (>180 -> True, <= 180 -> False)

Risposta 2

```
select s.SalesID, s.SaleDate, p.ProductName, p.ProductID, c.CategoryID, c.CategoryName, cc.CountryName, cc.CountryID, r.RegionName, r.RegionID, case when datediff(curdate(), s.SaleDate) > 180 then 'True' else 'False' end as Oltre180Giorni from Sales as s join Product as p on s.ProductID = p.ProductID join Category as c on p.CategoryID = c.CategoryID join Country as cc on s.CountryID = cc.CountryID join Region as r on cc.RegionID = r.RegionID;
```

SalesID	SaleDate	ProductName	ProductID	CategoryID	CategoryName	CountryName	CountryID	RegionName	RegionID	Oltre180Giorni
1	15/01/2023	Bici-100	1	1	Biciclette	Francia	1	EuropaOccidentale	1	True
2	22/03/2023	Bici-200	2	1	Biciclette	Italia	3	EuropaMeridionale	2	True
3	10/05/2023	Guanti M	3	6	Guanti	USA	7	AmericaDelNord	4	True
4	04/07/2023	Guanti L	4	6	Guanti	Germania	2	EuropaOccidentale	1	True
5	18/09/2023	Casco Pro	5	5	Caschi	Svezia	5	EuropaSettentrionale	3	True
6	08/01/2024	Casco Junior	6	5	Caschi	Brasile	9	AmericaDelSud	5	True
7	15/03/2024	Scarpa Strada	7	7	Scarpe	Grecia	4	EuropaMeridionale	2	True
8	20/05/2024	Scarpa Trail	8	7	Scarpe	Norvegia	6	EuropaSettentrionale	3	True
9	11/08/2024	Borsa Città	9	8	Borse	Giappone	10	AsiaOrientale	6	True
10	30/11/2024	Giacca Gara	10	2	Abbigliamento	Canada	8	AmericaDelNord	4	True

- 3) Esporre l'elenco dei prodotti che hanno venduto, in totale, una quantità maggiore della media delle vendite realizzate nell'ultimo anno censito. (ogni valore della condizione deve risultare da una query e non deve essere inserito a mano). Nel result set devono comparire solo il codice prodotto e il totale venduto.

Risposta 3

```
select s.ProductID, sum(s.Quantity) as TotaleVenduto
from Sales as s
where year(s.SaleDate) = (select max(year(s2.SaleDate)) from Sales as s2)
group by s.ProductID
having sum(s.Quantity) > (select avg(Quantity) from Sales where year(SaleDate) = (select
max(year(SaleDate)) from Sales));
```

	ProductID	TotaleVenduto
▶	6	12
	8	9

- 4) Esporre l'elenco dei soli prodotti venduti e per ognuno di questi il fatturato totale per anno.

Risposta 4

```
select s.ProductID, year(s.SaleDate) as Anno, sum(s.Quantity * s.UnitPrice) as FatturatoTotale
from Sales as s
group by s.ProductID, year(s.SaleDate)
order by s.ProductID, Anno;
```

- 5) Esporre il fatturato totale per stato per anno. Ordina il risultato per data e per fatturato decrescente.

Risposta 5

```
select c.CountryId, c.Countryname, year(s.SaleDate) as Anno, sum(s.Quantity * s.UnitPrice) as
FatturatoTotale
from Sales as s
join Country as c on c.CountryID = s.CountryID
group by s.CountryID, year(s.SaleDate)
order by Anno, FatturatoTotale desc;
```

CountryID	Countryname	Anno	FatturatoTotale
1	Francia	2023	5999.90
3	Italia	2023	3999.95
5	Svezia	2023	1199.92
7	USA	2023	599.80
2	Germania	2023	524.85
9	Brasile	2024	1559.88
8	Canada	2024	1399.93
6	Norvegia	2024	854.91
4	Grecia	2024	539.94
10	Giappone	2024	199.96

- 6) Rispondere alla seguente domanda: qual è la categoria di articoli maggiormente richiesta dal mercato?

Risposta 6

```

select c.CategoryID, c.CategoryName, count(s.SalesID) as NumeroTransazioni
from Sales as s
join Product as p on s.ProductID = p.ProductID
join Category as c on p.CategoryID = c.CategoryID
group by c.CategoryID, c.CategoryName
having count(s.SalesID) = (select max(NumeroTransazioni) from (select count(SalesID) as
NumeroTransazioni from Sales as s2 join Product as p2 on s2.ProductID = p2.ProductID group by
p2.CategoryID) as Conteggio);

```

CategoryID	CategoryName	NumeroTransazioni
1	Bidette	2
6	Guanti	2
5	Caschi	2
7	Scarpe	2

- 7) Rispondere alla seguente domanda: quali sono i prodotti invenduti? Proponi due approcci risolutivi differenti.

Risposta 7

```

. select p.ProductID, p.ProductName from Product as p where not exists (select 1 from Sales as s where
s.ProductID = p.ProductID);
. select p.ProductID, p.ProductName from Product as p where p.ProductID not in (select distinct
ProductID from Sales);

```

- 8) Creare una vista sui prodotti in modo tale da esporre una “versione denormalizzata” delle informazioni utili (codice prodotto, nome prodotto, nome categoria).

Risposta 8

```

create view Products as
(select ProductID, ProductName,CategoryName from Product as p
inner join Category as c on p.CategoryID = c.CategoryID);

```

- 9) Creare una vista per le informazioni geografiche

Risposta 9

```

create view InfoGeo as
(select c.CountryID, c.CountryName,r.RegionName from Country as c
inner join Region as r on c.RegionID = r.RegionID);

```

